

EDA DATA CLEANING

AIM

To perform exploratory data analysis on the California Housing Dataset by examining the dataset structure, dimensions, data types, statistical characteristics, missing values, duplicate records, housing-value distributions, ocean-proximity-wise housing patterns, housing-feature-wise relationships, geographical distributions, correlation analysis, interactive filters, parameter-based analysis, and creating an interactive dashboard for effective housing data analysis and decision-making.

ALGORITHM

1. Import the required libraries such as **Pandas, NumPy, Matplotlib, Seaborn, and ipywidgets**.
2. Load the **California Housing Dataset** into a Pandas DataFrame.
3. Display the **first five and last five records** of the dataset.
4. Display the **dataset information, shape, column names, and data types**.
5. Generate **descriptive statistical information** for numerical and categorical attributes.
6. Identify the **number and percentage of missing values** in each column.
7. Identify and remove **blank rows and duplicate records** from the dataset.
8. Perform **data cleaning and standardization** on the required categorical and numerical attributes.
9. Handle missing numerical values using appropriate **median imputation** and categorical missing values using the **mode**.
10. Analyze the distribution of **median house values** in the dataset.
11. Analyze the distribution of housing data across different **ocean proximity categories**.
12. Analyze housing characteristics using attributes such as **total rooms, total bedrooms, population, households, and median income**.
13. Analyze the relationship between **median income and median house value** and other important numerical attributes.
14. Analyze the **geographical distribution of house values** using latitude and longitude attributes.
15. Calculate and analyze the **correlation between numerical attributes** in the dataset.
16. Identify the **top 10 records based on median house value** and **top 10 records based on median income**.
17. Perform **outlier analysis** on the median house value using the Interquartile Range (IQR) method.
18. Create interactive analysis features including an **Ocean Proximity Filter, Minimum House Value Parameter, and an interactive Housing Analysis Dashboard**.
19. Export the **cleaned California Housing Dataset into CSV format** and display a confirmation message after successful file creation.
20. Display the **final cleaned dataset and verification results** and conclude the exploratory data analysis and data-cleaning process.

CODE

```
import os
import pandas as pd
import numpy as np
from IPython.display import display, clear_output
import ipywidgets as widgets

print("Required libraries imported successfully.")

path = "/content/housing prices.csv"

print("\nDataset path:", path)

df = pd.read_csv(path)

print("Dataset loaded successfully.")

print("\nFirst 5 rows:")
display(df.head())

print("\nLast 5 rows:")
display(df.tail())

print("\nDataset information:")
df.info()

print("\nShape:")
print(df.shape)

print("\nColumns:")
```

```
print(df.columns.tolist())
```

```
print("\nData types:")
```

```
print(df.dtypes)
```

```
print("\nStatistical description:")
```

```
display(df.describe())
```

```
print("\nCategorical column description:")
```

```
display(df.describe(include="object"))
```

```
print("\nMissing values:")
```

```
display(df.isnull().sum())
```

```
print("\nMissing value percentage:")
```

```
missing_percentage = df.isnull().sum() / len(df) * 100
```

```
display(missing_percentage.to_frame("Missing Percentage"))
```

```
print("\nNumber of completely blank rows:")
```

```
print(df.isnull().all(axis=1).sum())
```

```
print("\nNumber of duplicate records:")
```

```
print(df.duplicated().sum())
```

```
df = df.dropna(how="all")
```

```
df = df.drop_duplicates()
```

```
print("\nDataset after removing blank rows and duplicates:")
```

```
print(df.shape)
```

```
if "ocean_proximity" in df.columns:
```

```
    df["ocean_proximity"] = (  
        df["ocean_proximity"]  
        .astype(str)  
        .str.strip()  
        .str.upper()  
    )
```

```
numeric_columns = [
```

```
    "longitude",  
    "latitude",  
    "housing_median_age",  
    "total_rooms",  
    "total_bedrooms",  
    "population",  
    "households",  
    "median_income",  
    "median_house_value"
```

```
]
```

```
for column in numeric_columns:
```

```
    if column in df.columns:
```

```
        df[column] = pd.to_numeric(  
            df[column],  
            errors="coerce"  
        )
```

```
print("\nData cleaning and standardization completed.")
```

```
for column in numeric_columns:
```

```
if column in df.columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(
            df[column].median()
        )

categorical_columns = df.select_dtypes(
    include="object"
).columns

for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(
            df[column].mode()[0]
        )

print("\nMissing values handled successfully.")

print("\nMissing values after cleaning:")
display(df.isnull().sum())

print("\nDuplicate records after cleaning:")
print(df.duplicated().sum())

print("\nCleaned dataset shape:")
print(df.shape)

print("\nHousing value statistics:")

housing_value_statistics = df[
```

```
    "median_house_value"
].agg([
    "count",
    "mean",
    "median",
    "std",
    "min",
    "max"
])

display(
    housing_value_statistics.to_frame("Value")
)

print("\nOcean-proximity-wise housing analysis:")

ocean_analysis = (
    df.groupby("ocean_proximity")
    .agg(
        Number_of_Houses=(
            "median_house_value",
            "count"
        ),
        Average_House_Value=(
            "median_house_value",
            "mean"
        ),
        Average_Income=(
            "median_income",
            "mean"
        )
    )
)
```

```
)  
)  
.sort_values(  
    "Average_House_Value",  
    ascending=False  
)  
)  
  
display(ocean_analysis)  
  
print("\nHousing feature-wise statistics:")  
  
feature_analysis = df[  
    [  
        "total_rooms",  
        "total_bedrooms",  
        "population",  
        "households",  
        "median_income",  
        "median_house_value"  
    ]  
>].describe().T  
  
display(feature_analysis)  
  
print("\nCorrelation:")  
  
correlation = df.corr(  
    numeric_only=True  
)
```

```
display(correlation)
```

```
print("\nTop 10 houses by median house value:")
```

```
top_10_houses = (  
    df.sort_values(  
        "median_house_value",  
        ascending=False  
    )  
    .head(10)  
)
```

```
display(  
    top_10_houses[  
        [  
            "longitude",  
            "latitude",  
            "median_income",  
            "total_rooms",  
            "median_house_value",  
            "ocean_proximity"  
        ]  
    ]  
)
```

```
print("\nTop 10 records by median income:")
```

```
top_10_income = (  
    df.sort_values(  
        "median_income",  
        ascending=False  
    )  
    .head(10)  
)
```

```

        "median_income",
        ascending=False
    )
    .head(10)
)

display(
    top_10_income[
        [
            "longitude",
            "latitude",
            "median_income",
            "median_house_value",
            "ocean_proximity"
        ]
    ]
)

print("\nMinimum House Value Parameter:")

min_house_value = widgets.IntSlider(
    value=100000,
    min=int(df["median_house_value"].min()),
    max=int(df["median_house_value"].max()),
    step=10000,
    description="Minimum Value:",
    continuous_update=False
)

parameter_output = widgets.Output()

```

```
def minimum_value_analysis(change=None):  
    with parameter_output:  
        clear_output(wait=True)  
  
        selected_value = min_house_value.value  
  
        filtered_df = df[  
            df["median_house_value"] >= selected_value  
        ]  
  
        print(  
            "Minimum House Value:",  
            f"${selected_value:,.0f}"  
        )  
  
        print(  
            "Number of matching records:",  
            len(filtered_df)  
        )  
  
        print("\nAverage House Value:")  
  
        if len(filtered_df) > 0:  
            print(  
                f"${filtered_df['median_house_value'].mean():,.2f}"  
            )  
  
            print("\nAverage Median Income:")
```

```
print(  
    f'{filtered_df["median_income"].mean():.2f}'  
)
```

```
print("\nOcean Proximity Distribution:")
```

```
display(  
    filtered_df[  
        "ocean_proximity"  
    ]  
    .value_counts()  
    .to_frame("Number of Houses")  
)
```

```
else:  
    print("No records found.")
```

```
min_house_value.observe(  
    minimum_value_analysis,  
    names="value"  
)
```

```
display(  
    min_house_value,  
    parameter_output  
)
```

```
minimum_value_analysis()
```

```
print("\nInteractive Ocean Proximity Filter:")

ocean_options = [
    "ALL"
] + sorted(
    df["ocean_proximity"].unique().tolist()
)

ocean_filter = widgets.Dropdown(
    options=ocean_options,
    value="ALL",
    description="Location:"
)

filter_output = widgets.Output()

def filter_by_ocean(change=None):
    with filter_output:
        clear_output(wait=True)

        selected_location = ocean_filter.value

        if selected_location == "ALL":
            filtered_df = df.copy()
        else:
            filtered_df = df[
                df["ocean_proximity"] == selected_location
            ]

        print(
```

```
        "Selected Location:",
        selected_location
    )

    print(
        "Number of Records:",
        len(filtered_df)
    )

    print("\nFiltered Dataset:")

    display(
        filtered_df.head(10)
    )

ocean_filter.observe(
    filter_by_ocean,
    names="value"
)

display(
    ocean_filter,
    filter_output
)

filter_by_ocean()

print("\nInteractive Housing Analysis Dashboard:")

dashboard_location = widgets.Dropdown(
```

```
options=ocean_options,  
value="ALL",  
description="Ocean Proximity:"  
)
```

```
dashboard_value = widgets.IntSlider(  
    value=100000,  
    min=int(df["median_house_value"].min()),  
    max=int(df["median_house_value"].max()),  
    step=10000,  
    description="Minimum Value:",  
    continuous_update=False  
)
```

```
dashboard_output = widgets.Output()
```

```
def update_dashboard(change=None):  
    with dashboard_output:  
        clear_output(wait=True)  
  
        selected_location = dashboard_location.value  
        selected_value = dashboard_value.value  
  
        if selected_location == "ALL":  
            filtered_df = df.copy()  
        else:  
            filtered_df = df[  
                df["ocean_proximity"] == selected_location  
            ]
```

```
filtered_df = filtered_df[
    filtered_df["median_house_value"] >= selected_value
]

print("=" * 50)
print("CALIFORNIA HOUSING ANALYSIS DASHBOARD")
print("=" * 50)

print(
    "Ocean Proximity:",
    selected_location
)

print(
    "Minimum House Value:",
    f"${selected_value:,.0f}"
)

print(
    "Number of Records:",
    len(filtered_df)
)

if len(filtered_df) > 0:

    print(
        "\nAverage House Value:",
        f"${filtered_df['median_house_value'].mean():,.2f}"
    )
```

```
print(  
    "Average Median Income:",  
    f"{filtered_df['median_income'].mean():.2f}"  
)
```

```
print(  
    "Average Total Rooms:",  
    f"{filtered_df['total_rooms'].mean():.2f}"  
)
```

```
print("\nFiltered Records:")
```

```
display(  
    filtered_df.head(10)  
)
```

```
else:
```

```
print(  
    "\nNo records found for the selected conditions."  
)
```

```
dashboard_location.observe(  
    update_dashboard,  
    names="value"  
)
```

```
dashboard_value.observe(  
    update_dashboard,  
    names="value"  
)
```

```
display(  
    widgets.VBox([  
        dashboard_location,  
        dashboard_value  
    ])  
)  
  
display(dashboard_output)  
  
update_dashboard()  
  
output_file = "/content/housing_prices_cleaned.csv"  
  
df.to_csv(  
    output_file,  
    index=False  
)  
  
print("\nCSV file created successfully.")  
  
print(  
    "Cleaned dataset saved as:",  
    output_file  
)  
  
print("\n" + "=" * 60)  
print("RESULT")  
print("=" * 60)
```

```
print(  
    "Exploratory Data Analysis and Data Cleaning "  
    "were successfully performed on the California "  
    "Housing Dataset."  
)
```

```
print(  
    "The dataset structure, dimensions, data types, "  
    "statistical characteristics and missing values "  
    "were analyzed."  
)
```

```
print(  
    "Blank rows and duplicate records were removed "  
    "and the required data cleaning and standardization "  
    "were performed."  
)
```

```
print(  
    "Housing-value, ocean-proximity-wise and "  
    "feature-wise analyses were performed."  
)
```

```
print(  
    "Interactive filtering, minimum house value "  
    "parameter analysis and an interactive housing "  
    "analysis dashboard were created."  
)
```

```
print(  
    "The final interactive dashboard is available at "
```

```

    "The cleaned dataset was exported successfully "
    "in CSV format."
)

print("\nExperiment completed successfully.")

```

OUTPUT

```

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms        20433 non-null  float64
5   population             20640 non-null  float64
6   households             20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB

```

Statistical description:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000

Ocean-proximity-wise housing analysis:

..			
	Number_of_Houses	Average_House_Value	Average_Income
ocean_proximity			
ISLAND	5	380440.000000	2.744420
NEAR BAY	2290	259212.311790	4.172885
NEAR OCEAN	2658	249433.977427	4.005785
<1H OCEAN	9136	240084.285464	4.230682
INLAND	6551	124805.392001	3.208996

Housing feature-wise statistics:

	count	mean	std	min	25%	50%	75%	max
total_rooms	20640.0	2635.763081	2181.615252	2.0000	1447.7500	2127.0000	3148.00000	39320.0000
total_bedrooms	20640.0	536.838857	419.391878	1.0000	297.0000	435.0000	643.25000	6445.0000
population	20640.0	1425.476744	1132.462122	3.0000	787.0000	1166.0000	1725.00000	35682.0000
households	20640.0	499.539680	382.329753	1.0000	280.0000	409.0000	605.00000	6082.0000
median_income	20640.0	3.870671	1.899822	0.4999	2.5634	3.5348	4.74325	15.0001
median_house_value	20640.0	206855.816909	115395.615874	14999.0000	119600.0000	179700.0000	264725.00000	500001.0000

Correlation:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
longitude	1.000000	-0.924664	-0.108197	0.044568	0.069120	0.099773	0.055310	-0.015176	-0.045967
latitude	-0.924664	1.000000	0.011173	-0.036100	-0.066484	-0.108785	-0.071035	-0.079809	-0.144160
housing_median_age	-0.108197	0.011173	1.000000	-0.361262	-0.319026	-0.296244	-0.302916	-0.119034	0.105623
total_rooms	0.044568	-0.036100	-0.361262	1.000000	0.927058	0.857126	0.918484	0.198050	0.134153
total_bedrooms	0.069120	-0.066484	-0.319026	0.927058	1.000000	0.873535	0.974366	-0.007617	0.049457
population	0.099773	-0.108785	-0.296244	0.857126	0.873535	1.000000	0.907222	0.004834	-0.024650
households	0.055310	-0.071035	-0.302916	0.918484	0.974366	0.907222	1.000000	0.013033	0.065843
median_income	-0.015176	-0.079809	-0.119034	0.198050	-0.007617	0.004834	0.013033	1.000000	0.688075
median_house_value	-0.045967	-0.144160	0.105623	0.134153	0.049457	-0.024650	0.065843	0.688075	1.000000

Ocean Proximity Distribution:

Number of Houses	
ocean_proximity	
<1H OCEAN	8813
INLAND	3653
NEAR OCEAN	2442
NEAR BAY	2131
ISLAND	5

RESULT